

IRE Assignment-1, Component-1 Design Note

Lexical and Semantic Retrieval on EB-NeRD and MIND

Name: Naman Singhal **Roll Number:** 2024114013

1 What was built

A config-driven pipeline runs raw archives to metric reports in one command (`make data` for Q1's feature store, `make all` for everything) across **EB-NeRD demo** (50,080 impressions, 1,935 distinct users), **EB-NeRD small** (477,534 / 18,827) and **MIND-small** (230,117 / 94,057), counting all three splits. Both normalise into one schema (`impressions_*.parquet`: `id`, `user`, `timestamp`, `candidates[]`, `clicked[]`, `history[]`, and `articles.parquet`: `id`, `title`, `abstract`, `body`, `category`, `entities[]`), one code path rather than a Danish branch and an English one. `entities` normalises two source shapes into one `List(String)` (74%/87% non-empty).

Three scorers run over that schema, **bm25**, **emb**, **fused**, on two tracks: **re-rank** (score the pool the log actually showed, which is leaderboard-comparable) and **retrieval** (ignore the pool, pull the top-200 from the whole corpus, the "candidate generation to a few hundred" the spec asks for). The query for both is the user's most recent clicked titles, `history_len` of them: 30 on EB-NeRD, 100 on MIND, measured per dataset in Section 6. Section 7 covers the harness.

2 Temporal split

Both datasets ship their own train/test boundary, so I adopted it as `test_start` rather than inventing one, and carved `val` out of the earlier block by time (EB-NeRD small train/val/test: 168,522/64,365/244,647, and MIND-small: 95,071/61,894/73,152 impressions). EB-NeRD's day boundary is **07:00**, not midnight, a detail that only shows up in the data.

EB-NeRD ships a separate `history.parquet` per block whose window ends exactly where its impressions begin, so the data gives me the behaviour-window boundary rather than my guessing it (the alternative, all 7 provided train days with validation halved, buys ~38% more train data at the cost of 7-day-stale test history, and nothing here is trained, so fresh history wins).

Enforcement, not discipline. `split.py` asserts on every build, failing rather than warning: `strict train < val < test`, history-precedes-impression, `clicked` a subset of `candidates`, every candidate resolves. Where impossible it says so: MIND's history has no

per-item timestamps, so that guarantee is structural rather than verified, not vacuous.

`tests/test_leakage.py` (`make test`, 19 tests, ~20s) re-checks the same properties from *outside*, reading only `data/processed/`: the build asserts what it meant to write, the tests assert what is on disk, plus two checks that exist only there (`datasets.yaml` cut-offs still match the artifacts, and no module outside `eval/run.py` mentions a serving-unavailable column). I checked the suite is not vacuous by injecting a future click and watching it fire.

3 Lexical axis

bm25s over title+abstract with a per-dataset Snowball stemmer (`rank_bm25` scores one query at a time in pure Python, unusable at 65k documents, while Pyserini/Lucene scales further but drags in a JVM).

Stemming is not free, and not what I assumed. The brief specifies no preprocessing, so this was my choice. On val, `stem - none` is **+0.0019** [+0.0007, +0.0029] **AUC on MIND, zero on EB-NeRD, and significantly negative on recall@200 for both** (-0.0020, -0.0023): it trades precision for variant-matching, worth it ordering ~8 candidates, harmful picking 200 from 20,738. I keep it as right for the headline metric, a track-dependent trade. Danish compound splitting from corpus vocabulary fired on 15% of tokens and made both **worse**: decompounding needs a lexicon, not statistics.

The finding that mattered. EB-NeRD first scored AUC **0.5035**, random. Rather than report that, I checked whether it was a bug: an **oracle test** (query = the clicked article's own title) ranked it #1 in 300/300 impressions at AUC 0.9990, and a history-length sweep was flat, ruling out misalignment and query dilution. MIND worked at 0.58, and that asymmetry located the cause: **bm25s ships no Danish stopword list**, so thirty concatenated Danish titles were mostly function words. The Snowball list moved EB-NeRD to **0.5232**. A missing stopword list is silent, it degrades without erroring.

4 Semantic axis

MIND encodes title+abstract with **all-MiniLM-L6-v2** (**11m16s**, cached once, 65,238-article union across

its two files). I build each user vector as the L2-normalised mean of their last `history_len` history vectors (100 on MIND, 30 on EB-NeRD, Section 6) and score with cosine over a FAISS IndexFlatIP, exact at this scale. IVF/PQ waits for the 10 $\times$ analysis.

The encoder is a measurement, not a preference. Q3 allows the provided article vectors *or* your own from BERT/XLM-RoBERTa, so I ran the identical pipeline on each candidate: same splits, same BM25, same fusion, only the article vectors differ. I select on **val**, because selecting on test would tune the choice to the number I then report.

On shipping contrastive_vector: it is an official EB-NeRD artifact from the same `artifacts/` folder as the other two, so it sits in Q3’s “provided article embeddings” arm, word2vec and mBERT are that arm’s named examples, not its full extent. It is the strongest EB-NeRD encoder I measured, beating my own XLM-R by +0.0197 val AUC, a gap well outside both intervals.

One property predicts all four: **did training make cosine distance mean topical similarity?** Contrastive training optimises exactly that and wins, XLM-R paraphrase fine-tuning approximates it and comes second, word2vec (predict neighbouring words) does not, and raw mBERT (fill in blanks) emphatically does not, its vectors sitting in a narrow cone where everything looks alike. Size predicts nothing: the 300-d model beats one 768-d model. Fusion detects the mBERT failure unprompted, tuning alpha to 0.00 on demo, discarding the semantic axis outright.

MIND kept MiniLM, and that is the sharper lesson. Against the same XLM-R, MiniLM wins on **val** (0.6338 vs 0.6256) and *loses* on **test** (0.6339 vs 0.6364), a real reversal, near-disjoint intervals. I fixed the rule before looking, so MiniLM stays. “Take the better test number” would have flipped it, costing 0.0025 test AUC to obey, the entire point of having a rule. An English specialist beats a 50-language generalist on English, while that generalist beats a 2013 word-vector model on Danish.

A four-way MIND ablation settles the accuracy-vs-cost question. MiniLM (384-d, 0.6356) ties `mpnet` (768-d, 0.6344, not significant) and beats `e5` (0.6067) and `bge` (0.5899) outright, at half the FAISS index size and half the per-query cosine cost: no trade-off to weigh, it wins on both axes. That `bge`, built for retrieval, loses by 0.0457 to a small general-purpose model is the clearest evidence that neither dimensionality nor stated training purpose predicts performance here, only measurement does.

5 Candidate generation: recall@K, and a split-dependent winner

Recall@K over the **full catalogue**, not the pool the log showed, measures candidate generation: the share of an impression’s clicked articles found in the top K. Cold-start impressions retrieve nothing and I score them 0 rather than excluding them, since a retriever that cannot serve a user has failed for that user.

On EB-NeRD the retrieval winner flips between val and test, both directions significant: BM25 wins val, embeddings win test, same code, same K, a sign change with disjoint intervals, not noise. MIND shows no such flip. Section 6 finds the identical pattern in the fusion weight, pointing at the dataset rather than either method: val is a 2-day window against test’s 7, each with its own history window, so the relationship genuinely drifts.

Re-ranking and retrieving stay different jobs, by margin now, not direction. The same contrastive vectors beat BM25 by +0.0289 AUC ordering the pool and by only +0.0030 recall@200 searching all 20,738 articles, a 10 $\times$ gap for the same encoder and split, evidence for different signals per stage. Absolute recall is low everywhere (2–5%), Section 8 explains why, a property of the corpus, not the retriever.

6 Fusion, an addition the brief did not ask for

Q3 asks only to *compare* lexical and semantic. This third system is mine. BM25 scores and cosines are on different scales, and BM25’s moves with query length, so I z-normalise both **within each candidate pool** and mix them as $\alpha \cdot \text{emb} + (1 - \alpha) \cdot \text{bm25}$, tuning α on **val** and applying it unchanged to test.

On EB-NeRD fusion wins on val and significantly loses on test, on both bundles: the alpha chosen on val does not generalise across the boundary. This is the same reversal Section 5 finds in retrieval, reached through a different measurement, a property of EB-NeRD, not an artefact of either method.

Consequently **the reported EB-NeRD system is embeddings alone**, and only MIND ships fused. Fusion still earns its place: it *detected* the instability, and on MIND it is a real if small win. Judged on val alone, the tempting shortcut, I would have shipped fusion on all three and been wrong on two.

history_len was shared across datasets and shouldn’t have been. A sweep on MIND val found the constant (30) truncating a real tail: MIND’s median history is 20, not EB-NeRD’s 258, so 30 discarded signal rather than noise. AUC rose significantly to 100 clicks (bm25 +0.0039, emb +0.0018) then plateaued exactly there. EB-NeRD’s own sweep (Section 3) was flat over the same range, so it stays at 30, and his-

Encoder	Arm	Dim	Val AUC	vs BM25 (0.5205)
Ekstra_Bladet_contrastive_vector (<i>ships</i>)	provided	768	0.5506 [.5480, .5531]	beats
paraphrase-multilingual-mpnet (XLM-R)	own	768	0.5309 [.5282, .5336]	beats
Ekstra_Bladet_word2vec	provided	300	0.5098 [.5073, .5124]	loses
google_bert_base_multilingual_cased	provided	768	0.4857 [.4832, .4880]	below random

Table 1: Semantic-only AUC on ebnerd_small val, everything else held fixed.

recall@200	bm25	emb	emb – bm25
EB-NeRD small, val	0.0214	0.0198	−0.0016 [−.0030, −.0002]
EB-NeRD small, test	0.0247	0.0277	+0.0030 [+0.0022, +.0039]
MIND-small, test	0.0220	0.0239	+0.0019 [+0.0008, +.0031]

Table 2: Full-corpus retrieval, small bundles, and demo tracked these within noise (Section 8).

tory_len is now per-dataset in config. This table reflects the corrected MIND value.

A second MIND-only signal, entity overlap, ships alongside emb. MIND’s entities are clean and Wikidata-linked, and a candidate’s Jaccard overlap with the user’s recent-history entities, blended onto emb ($\alpha \cdot \text{entity} + (1 - \alpha) \cdot \text{emb}$). I selected it on val (0.20) and checked it once on test: **+0.0022** [+0.0018, +0.0026] **AUC**, holding almost exactly (val +0.0023), unlike the fusion alpha above. I did not try it on EB-NeRD, whose entities are a different extraction (ner_clusters) with different coverage.

7 Evaluation harness

The harness computes every metric **per impression**: slicing masks that vector, and bootstrapping resamples it over impressions, the approximately-independent unit. Comparisons use a **paired** bootstrap on shared resamples, since two systems on the same impressions have correlated errors, an unpaired comparison of independent intervals is far too conservative.

Cold-start needed a defensible definition.

“Empty history” fails on EB-NeRD (zero such users, median 258), so the harness uses a per-dataset bottom-decile threshold plus the true zero-history slice where one exists. MIND’s scores 0.5125 identically across systems, correctly, no history means no ranking invented.

Beyond-accuracy over the top-10: category intra-list diversity, novelty as self-information against *train* click shares, coverage, sobering at **5–21%** of the catalogue ever surfaced.

8 Observations

Content signals are weak here, and that is the result, not a failure. MIND reaches 0.6391 (Section 6), within reach of published neural baselines for

MIND-small (~ 0.65 – 0.67), unsupervised. EB-NeRD tops out at 0.5397: pools are much smaller (median 8–9 vs MIND’s 23–26) and drawn from a tight recency window, so candidates are already plausible, little for content to separate.

Simple behavioural features are not the shortcut they look like. Train-click popularity scores *below* random on EB-NeRD test (AUC 0.4498 [0.4485, 0.4510] on small, 0.4742 on demo): the pool the log showed is already popularity-filtered, so within it, being popular is mildly *anti*-predictive.

Demo was representative. Every system landed within 0.002 of its demo value at $10\times$ the users. The extra data bought precision, CIs tightened from ± 0.0040 to ± 0.0013 , deciding fusion.

The retrieval track is weak regardless of scorer (recall@200 of 2–5%): EB-NeRD’s corpus spans 1998–2023 while impressions are one week in May 2023, and the publication filter removes only *future* articles, not stale ones. A recency window on the retrievable corpus is the obvious C2 lever.

9 Anti-gaming and serving availability

total_inviews/total_pageviews/total_read_time are lifetime aggregates over the whole collection period, embedding the future, so serving time cannot compute them. I carry them into the corpus **only** so the harness can quantify them, and no shipped scorer reads them. Test AUC without/with popularity: EB-NeRD demo 0.5384 $\rightarrow$ 0.5793 (+0.0409 [+0.0380, +0.0437]), EB-NeRD small 0.5380 $\rightarrow$ 0.5797 (+0.0417 [+0.0408, +0.0426]).

One serving-unavailable feature is worth +0.042, still around $1.4\times$ the entire semantic axis (+0.0289 emb over bm25, Section 6), the largest legitimate win here. Any comparison quietly including it is not measuring the same system.

I enforce this rather than trusting it: the split drops post-click engagement columns, and retrieval fil-

AUC	bm25	emb	fused	α	fused – emb
EB-NeRD small, val	0.5205	0.5506	0.5528	0.70	+0.0022 [+0.0010, +0.0033]
EB-NeRD small, test	0.5107	0.5397	0.5380	0.70	−0.0017 [−0.0023, −0.0011]
MIND-small, test	0.5685	0.6369	0.6381	0.75	+0.0012 [+0.0005, +0.0018]

Table 3: Fusion against its components. Demo tracked these within noise (Section 8).

ters `published_time <= impression_time` (867 EB-NeRD articles postdate the test window).

10 Where it breaks at $10\times$, measured, not projected

The leaderboard submission made this section measured, not hypothetical: Codabench scores `ebnerd_testset` at **13.5M impressions, 205,925,868 candidate pairs, $28\times$ `ebnerd_small`**.

Memory is the wall, and it is one specific idiom. Three separate times the break was `.to_list()` pulling Arrow into boxed Python objects: `np.vstack(col.to_list())` peaks at **5.8 GB to build a 385 MB array**, the same idiom OOM-killed the harness at 10 GB RSS adding `recall@K.explode().to_numpy()` plus Polars set intersections fixed all three, byte-identically.

Streaming works and costs little. `pipeline/submit.py` scores in 200k-impression slices: peak RSS stays **flat against dataset size, not linear**, in **13m30s** (33m28s at the same peak under the earlier XLM-R build), a throughput of $\sim 16.7k$ impressions/s. MIND-large’s 2.4M took 4m38s at 2.0 GB. Slice pushdown reaches the parquet row groups, so an offset-13M slice costs the same as one at 0, and without it streaming would be $O(n^2)$. MIND needed a one-off TSV $\rightarrow$ parquet conversion, CSV has no such pushdown and OOM-killed the first attempt.

BM25 is expensive, but less so than its operation count implies. `get_scores` is dense over the corpus per query, roughly 2M MIND-large histories $\times$ 120,961 articles. That $\sim 10^{11}$ figure looks impractical on paper, so I timed it rather than let the estimate stand: 1.6ms/query, or **~ 1.6 h**, roughly $7\times$ the embedding run. Slow, not a wall. Operation counts are not wall-clock for a sparse vectorised kernel, which is why the estimate needed a measurement. The fix is unchanged: a dense per-query scorer is the wrong shape, WAND-style early termination removes the cost. At `ebnerd_large` (600M) FAISS flat is $O(N\cdot d)/\text{query}$, needing IVF-PQ with `nlist` $\sim \sqrt{N}$.

11 Leaderboard submissions

Both competitions score held-out sets far larger than the splits above (`ebnerd_testset` 13.5M impressions,

MIND-large test 2.4M), which `pipeline/submit.py` streams (Section 10) with unchanged scoring semantics. **Both entries are embeddings-only, not fused:** on EB-NeRD that *is* the reported system (Section 6). On MIND, BM25 would cost ~ 1.6 h for +0.0012 AUC, a deliberate trade reported as a different system.

	Sub.	AUC	MRR	n@5	n@10	Rk
EB-NeRD	895220	.5381	.3521	.3868	.4669	138
MIND	898179	.6503	.3198	.3454	.4010	36

Table 4: Final Codabench submissions (comps. 2469, 13967), organisers’ own output. Codabench username: `namsyn`.

The finals supersede EB-NeRD 893893 (0.5336, XLM-R) and MIND 892088 (0.6460, `history_len=30`, no entity blend).

Each score reads against the offline number for the system that produced it. 895220 (post-switch) against offline `ebnerd_demo emb test`, 0.5418: leaderboard -0.0037 , unremarkable, a larger independent population regressing toward the mean. 898179 against offline `emb+entity test`, 0.6391: leaderboard $+0.0112$, the reassuring direction. (An earlier draft mis-mapped 893893 to the wrong encoder here, dropped rather than repeated.)

MIND’s MRR is the one metric that moved against us, 0.3198 official against 0.3548 offline for the same `emb+entity` system, while AUC and both nDCGs rose, consistently across every MIND submission: a definitional gap, not noise. `eval/metrics.py` takes the *first* clicked article’s reciprocal rank, while averaging over *all* clicks scores multi-click impressions lower while leaving AUC and nDCG intact. I cannot verify this without the organisers’ scorer, so I record rather than resolve it.

I validated both before upload: every line a correct-length rank permutation in test-file row order, plus 40 impressions re-ranked independently in float64. I exclude 897967 (MIND, 0.5012), a `-limit` smoke test that overwrote the real file. **Evidence:** `leaderboard_screenshots/` and `{ebnerd,mind}_scoring_result/`, both under `deliverable/`. `docs/NOTES.md` records every measurement quoted here, plus the ablations and failures behind them.